

dsbda-lab-ass-6

April 3, 2026

1 Assignment 6

1.1 Import Required Library

```
[1]: import pandas as pd
```

1.2 Read iris dataset into Dataframe

```
[2]: df = pd.read_csv('iris.csv')
```

```
[3]: df.shape
```

```
[3]: (150, 6)
```

```
[4]: df.head()
```

```
[4]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

1.3 Define x and y variables

```
[5]: #input data
x=df.drop('Species',axis=1)

#output data
y=df['Species']
```

```
[6]: y.value_counts()
```

```
[6]: Species
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
```

Name: count, dtype: int64

1.4 train-test split

```
[7]: from sklearn.model_selection import train_test_split
```

```
[8]: x_train,x_test,y_train,y_test=train_test_split(x,y,random_state=0,test_size=0.  
    ↪25)
```

```
[9]: x_train.shape
```

```
[9]: (112, 5)
```

```
[10]: x_test.shape
```

```
[10]: (38, 5)
```

1.5 Apply Regression Model

```
[11]: #import the class  
from sklearn.naive_bayes import GaussianNB
```

```
[12]: #create the object  
clf= GaussianNB()
```

```
[13]: #train the algorithm  
clf.fit(x_train,y_train)
```

```
[13]: GaussianNB()
```

```
[14]: y_pred=clf.predict(x_test)
```

```
[15]: y_pred.shape
```

```
[15]: (38,)
```

1.6 Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall

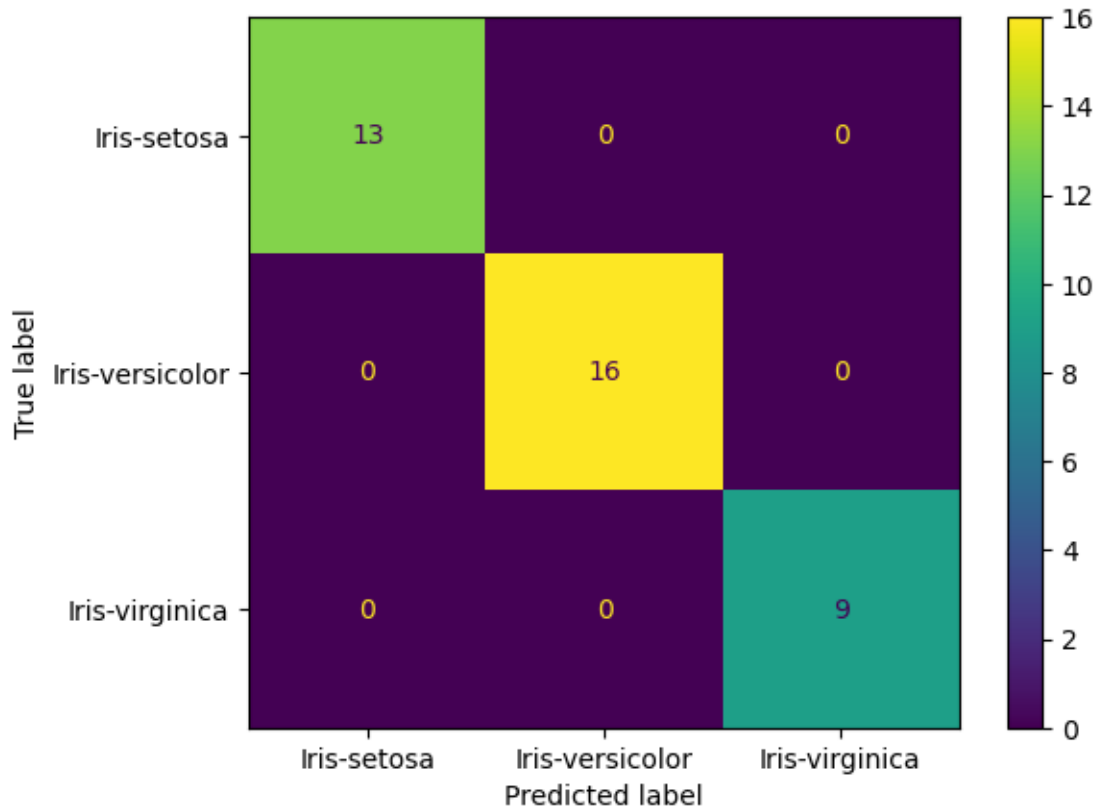
```
[16]: from sklearn.metrics import confusion_matrix  
from sklearn.metrics import classification_report  
from sklearn.metrics import accuracy_score  
from sklearn.metrics import ConfusionMatrixDisplay
```

```
[17]: confusion_matrix(y_test,y_pred)
```

```
[17]: array([[13,  0,  0],
           [ 0, 16,  0],
           [ 0,  0,  9]], dtype=int64)
```

```
[18]: ConfusionMatrixDisplay.from_estimator(clf, x_test, y_test)
```

```
[18]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1a7ecb4fc20>
```



```
[19]: accuracy_score(y_test,y_pred)
```

```
[19]: 1.0
```

```
[20]: clf.predict_proba(x_test)
```

```
[20]: array([[8.68564082e-238, 4.76522327e-008, 9.99999952e-001],
         [2.26575230e-085, 1.00000000e+000, 2.82780434e-010],
         [1.00000000e+000, 3.46164738e-020, 1.01145154e-035],
         [1.67840419e-316, 1.02626618e-007, 9.99999897e-001],
         [1.00000000e+000, 4.97375377e-021, 4.68110614e-040],
         [0.00000000e+000, 6.30238467e-011, 1.00000000e+000],
         [1.00000000e+000, 1.26427628e-017, 2.26981968e-033],
```

```
[9.51059698e-152, 9.88268650e-001, 1.17313499e-002],
[8.45240100e-155, 9.99573841e-001, 4.26159071e-004],
[5.94731541e-096, 9.9999845e-001, 1.54965299e-007],
[2.27917620e-225, 3.25039440e-004, 9.99674961e-001],
[4.20930008e-133, 9.99999170e-001, 8.29738436e-007],
[9.89228245e-137, 9.99997614e-001, 2.38551672e-006],
[2.13374320e-141, 9.99998486e-001, 1.51407682e-006],
[3.29098703e-143, 9.99997279e-001, 2.72092881e-006],
[1.00000000e+000, 2.40543856e-018, 9.34801856e-035],
[7.08731706e-135, 9.99914946e-001, 8.50537101e-005],
[9.66632821e-117, 9.99993696e-001, 6.30421414e-006],
[1.00000000e+000, 6.80649992e-016, 1.30377606e-031],
[1.00000000e+000, 5.79992580e-019, 2.03721824e-036],
[6.35957322e-195, 9.21493092e-005, 9.99907851e-001],
[2.77478070e-132, 9.99996523e-001, 3.47650087e-006],
[1.00000000e+000, 1.90478001e-015, 1.34491074e-032],
[1.00000000e+000, 6.73294477e-021, 2.13052670e-040],
[1.30670787e-178, 4.65617767e-004, 9.99534382e-001],
[1.00000000e+000, 4.14333384e-021, 9.61925219e-039],
[1.00000000e+000, 1.77413231e-011, 4.66180038e-026],
[2.04327535e-116, 9.99653632e-001, 3.46368441e-004],
[2.78665763e-051, 9.99999993e-001, 7.30501053e-009],
[1.00000000e+000, 9.02172765e-016, 9.75375573e-033],
[1.29291440e-238, 2.20724591e-007, 9.99999779e-001],
[1.53347802e-133, 9.99837090e-001, 1.62910166e-004],
[1.00000000e+000, 6.64953350e-019, 8.14099428e-036],
[1.12741612e-185, 1.77204245e-004, 9.99822796e-001],
[1.96987714e-272, 6.29180680e-010, 9.99999999e-001],
[6.52805095e-091, 9.99999996e-001, 4.49363003e-009],
[1.00000000e+000, 1.30410954e-016, 1.12844624e-033],
[1.44076713e-183, 9.83884894e-001, 1.61151057e-002]])
```

```
[21]: print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	13
Iris-versicolor	1.00	1.00	1.00	16
Iris-virginica	1.00	1.00	1.00	9
accuracy			1.00	38
macro avg	1.00	1.00	1.00	38
weighted avg	1.00	1.00	1.00	38

1.7 Make predictions

```
[22]: new1=[[5.5,3.1,1.0,0.8,4]]
      clf.predict(new1)[0]
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X
does not have valid feature names, but GaussianNB was fitted with feature names
warnings.warn(
```

```
[22]: 'Iris-virginica'
```

```
[23]: new1=[[4.5,2.9,3.1,0.4,3]]
      clf.predict(new1)[0]
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X
does not have valid feature names, but GaussianNB was fitted with feature names
warnings.warn(
```

```
[23]: 'Iris-versicolor'
```

```
[24]: new1=[[6.5,3.3,4.9,1.8,1]]
      clf.predict(new1)[0]
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X
does not have valid feature names, but GaussianNB was fitted with feature names
warnings.warn(
```

```
[24]: 'Iris-setosa'
```